MARTHANDAM COLLEGE OF

ENGINEERING AND TECHNOLOGY

Kuttakuzhi

Veeyannoor Post

(Near Marthandam)

Kanyakumari District

RECORD NOTEBOOK

AD3511 – DEEP LEARNING LABORATORY

NAME: …….....

ROLL NO: …….

MARTHANDAM COLLEGE OF ENGINEERING AND TECHNOLOGY

Kuttakuzhi

Veeyannoor Post

(Near Marthandam)

Kanyakumari District

Certified that this is the bonafide record of work done by Mr./Ms.…….……of the second semester in ……..Department of this college in the **AD3511 – Deep Learning Laboratory** during 2026–27 in partial fulfillment of the requirement of the B.E Degree course of the Anna University Chennai.

Staff-in-charge Head of the Department

University Reg. No: ……

University Exam held in: …….

Internal Examiner External Examiner

INDEX

EX NO	DATE	TITLE	MARKS	SIGN
1.		SOLVING XOR PROBLEM USING DNN		
2.		CHARACTER RECOGNITION USING CNN		
3.		FACE RECOGNITION USING CNN		
4.		LANGUAGE MODELING USING RNN		
5.		SENTIMENT ANALYSIS USING LSTM		
6.		PARTS OF SPEECH TAGGING USING SEQUENCE TO SEQUENCE ARCHITECTURE		
7.		MACHINE TRANSLATION USING ENCODER-DECODER MODEL		
8.		IMAGE AUGMENTATION USING GANS		

EXP NO	SOLVING XOR PROBLEM USING DNN	DATE
1		

AIM:

To write a Python program to solve the XOR problem using DNN.

ALGORITHM:

Step 1: Give the input values and the correct output for the XOR problem.

Step 2: Set up a small neural network with one hidden layer.

Step 3: Start with some random weights and biases.

Step 4: Repeat the following steps many times (training):

Step 5: Do a forward pass: Use the inputs to calculate the output of the network.

Step 6: Find the error: Compare the network's output with the correct answer.

Step 7: Adjust the weights and biases to reduce the error.

Step 8: Save the error (called cost) after each round.

Step 9: After training, draw a graph showing how the error went down.

Step 10: Test the network with the inputs again and check if the answers are close to correct.

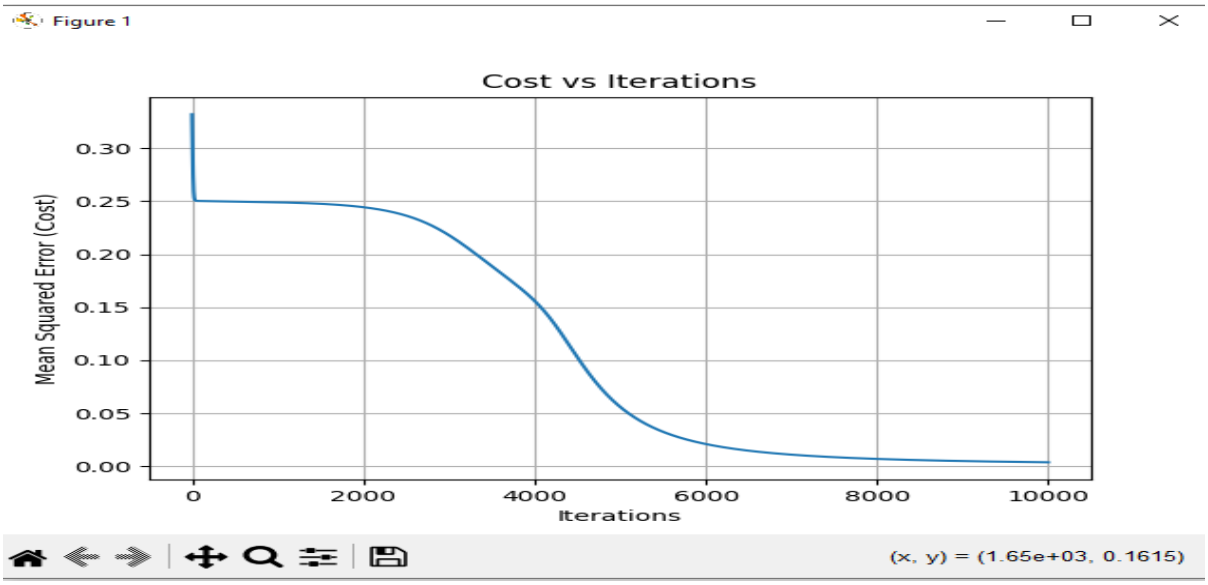
PROGRAM:

```
import numpy as np
import matplotlib.pyplot as plt
def sigmoid(x):
   return 1 / (1 + np.exp(-x))
def sigmoid_derivative(x):
   return x * (1 - x)
X = np.array([[0, 0],
              [0, 1],
```

```
            [1, 0],
            [1, 1]])
y = np.array([[0], [1], [1], [0]])
np.random.seed(42)
input_layer_neurons = 2
hidden_layer_neurons = 4
output_neurons = 1
wh = np.random.uniform(size=(input_layer_neurons, hidden_layer_neurons))
bh = np.random.uniform(size=(1, hidden_layer_neurons))
wo = np.random.uniform(size=(hidden_layer_neurons, output_neurons))
bo = np.random.uniform(size=(1, output_neurons))
epochs = 10000
learning_rate = 0.1
cost_history = []
for epoch in range(epochs):
    hidden_input = np.dot(X, wh) + bh
    hidden_output = sigmoid(hidden_input)
    final_input = np.dot(hidden_output, wo) + bo
    final_output = sigmoid(final_input)
    error = y - final_output
    cost = np.mean(np.square(error))  # MSE
    cost_history.append(cost)
    d_output = error * sigmoid_derivative(final_output)
    error_hidden = d_output.dot(wo.T)
    d_hidden = error_hidden * sigmoid_derivative(hidden_output)
    wo += hidden_output.T.dot(d_output) * learning_rate
    bo += np.sum(d_output, axis=0, keepdims=True) * learning_rate
    wh += X.T.dot(d_hidden) * learning_rate
    bh += np.sum(d_hidden, axis=0, keepdims=True) * learning_rate
plt.plot(cost_history)
plt.title('Cost vs Iterations')
```

```
plt.xlabel('Iterations')
plt.ylabel('Mean Squared Error (Cost)')
plt.grid(True)
plt.show()
print("Final Predictions:")
print(np.round(final_output, 4))
```

OUTPUT:



Final Predictions:

[[0.0469]

[0.9566]

[0.9255]

[0.0718]]

RESULT:

EXP NO	CHARACTER RECOGNITION USING CNN	DATE
2		

AIM:

To write a python program for character recognition using CNN.

ALGORITHM:

Step 1: Start the program.
Step 2: Import necessary libraries: torch, torch.nn, torch.optim, torchvision, matplotlib.pyplot.
Step 3: Load the EMNIST dataset with transformations (tensor conversion and normalization). Split into training and testing sets and create data loaders.
Step 4: Define the CNN model with two convolutional layers, max pooling, a flatten layer, and two fully connected layers. Include ReLU activations and dropout for regularization.
Step 5: Initialize the model, set the device (CPU or GPU), define loss function (CrossEntropyLoss) and optimizer (Adam).
Step 6: Train the model over a few epochs: loop over training batches, move data to device, perform forward pass, calculate loss, backpropagate, and update weights.
Step 7: Evaluate the model on the test dataset: loop over test batches, move data to device, predict labels, and calculate accuracy.
Step 8: Display a sample test image and its predicted label using matplotlib.
Step 9: End the program.

PROGRAM:

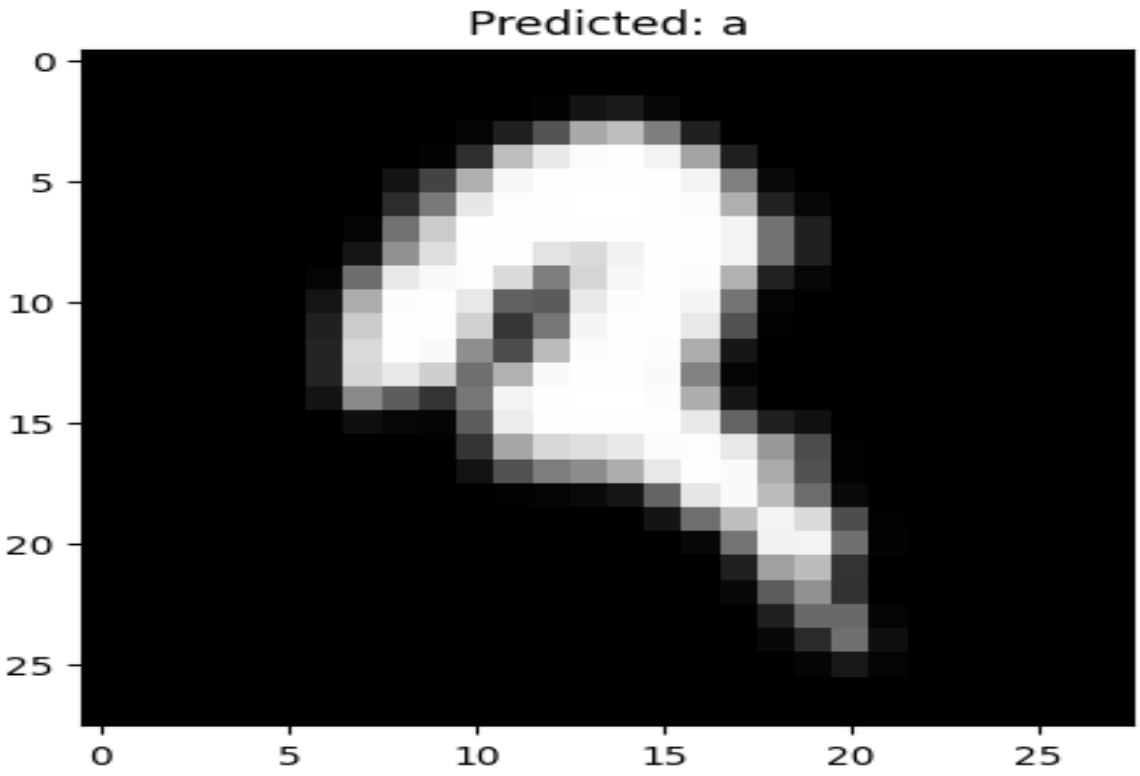
```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
```

```
import matplotlib.pyplot as plt
transform = transforms.Compose([transforms.ToTensor(), transforms.Normalize((0.5,), (0.5,))])
trainset = torchvision.datasets.EMNIST("./data", split="letters", train=True, download=True,
transform=transform)
testset  = torchvision.datasets.EMNIST("./data", split="letters", train=False, download=True,
transform=transform)
trainloader = torch.utils.data.DataLoader(trainset, batch_size=128, shuffle=True)
testloader  = torch.utils.data.DataLoader(testset, batch_size=128)
class CNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(1, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2,2),
            nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2,2),
            nn.Flatten(),
            nn.Linear(64*7*7, 128), nn.ReLU(),
            nn.Linear(128, 27)  # 27 classes
        )
    def forward(self, x):
        return self.net(x)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = CNN().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
for epoch in range(3):
    for imgs, labels in trainloader:
        imgs, labels = imgs.to(device), (labels-1).to(device)
        optimizer.zero_grad()
        loss = criterion(model(imgs), labels)
        loss.backward()
        optimizer.step()
```

```
    print(f"Epoch {epoch+1} complete")
correct, total = 0, 0
with torch.no_grad():
    for imgs, labels in testloader:
        imgs, labels = imgs.to(device), (labels-1).to(device)
        preds = model(imgs).argmax(1)
        correct += (preds==labels).sum().item()
        total += labels.size(0)
print(f"Accuracy: {100*correct/total:.2f}%")
imgs, labels = next(iter(testloader))
pred = model(imgs[0].unsqueeze(0).to(device)).argmax(1).item()
plt.imshow(imgs[0].squeeze(), cmap="gray")
plt.title("Predicted: " + chr(pred + ord('a')))
plt.show()
```

OUTPUT

100%|██████████| 562M/562M [00:08<00:00, 66.1MB/s]
Epoch 1 complete
Epoch 2 complete
Epoch 3 complete
Accuracy: 92.64%



RESULT:

EXP NO	FACE RECOGNITION USING CNN	DATE
3		

AIM:

To write a python program for face recognition using CNN.

ALGORITHM:

Step 1: Start the program.

Step 2: Load the LFW dataset with a minimum number of faces per person, normalize pixel values to the range [0,1], and reshape the images to have a single channel.

Step 3: Split the dataset into training and testing sets, and convert them into PyTorch DataLoaders for batch processing.

Step 4: Define the CNN model with two convolutional layers followed by max-pooling layers, a dropout layer to reduce overfitting, flatten the output dynamically for the fully connected layer, and fully connected layers to output class probabilities.

Step 5: Set the device to GPU if available, otherwise CPU, and initialize the model, the cross-entropy loss function, and the Adam optimizer.

Step 6: Train the model for a fixed number of epochs by performing a forward pass, computing the loss, backpropagating, and updating the weights for each batch, and print the average loss per epoch.

Step 7: Evaluate the model on the test set by disabling gradient computation, performing a forward pass, computing predictions, comparing them with true labels, and calculating the test accuracy.

Step 8: Visualize predictions for a few test images by taking the first five images from the test set, predicting their classes, and plotting the images with predicted and true labels.

Step 9: End the program.

PROGRAM:

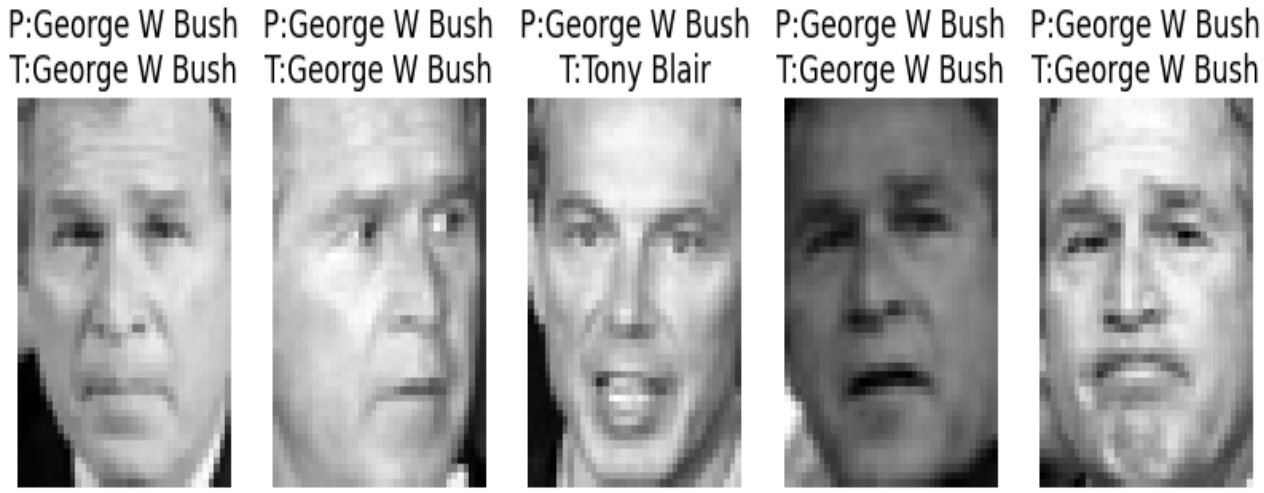
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_lfw_people
from sklearn.model_selection import train_test_split
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
faces = fetch_lfw_people(min_faces_per_person=70, resize=0.4)
X, y, target_names = faces.images / 255.0, faces.target, faces.target_names
X = X.astype(np.float32).reshape(-1, 1, X.shape[1], X.shape[2])
X_train, X_test, y_train, y_test = train_test_split(
    torch.tensor(X), torch.tensor(y), test_size=0.25, random_state=42
)
train_loader = DataLoader(TensorDataset(X_train, y_train), batch_size=32, shuffle=True)
test_loader  = DataLoader(TensorDataset(X_test, y_test), batch_size=32)
class CNN(nn.Module):
    def __init__(self, n_classes):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 32, 3, 1)
        self.conv2 = nn.Conv2d(32, 64, 3, 1)
        self.pool  = nn.MaxPool2d(2, 2)
        self.drop  = nn.Dropout(0.5)
        with torch.no_grad():
            dummy = torch.zeros(1, 1, X.shape[2], X.shape[3])
            dummy = self.pool(torch.relu(self.conv1(dummy)))
            dummy = self.pool(torch.relu(self.conv2(dummy)))
            flat_size = dummy.numel()
```

```
        self.fc1 = nn.Linear(flat_size, 128)
        self.fc2 = nn.Linear(128, n_classes)
    def forward(self, x):
        x = self.pool(torch.relu(self.conv1(x)))
        x = self.pool(torch.relu(self.conv2(x)))
        x = x.view(x.size(0), -1)
        x = torch.relu(self.fc1(x))
        x = self.drop(x)
        x = self.fc2(x)
        return x
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = CNN(len(target_names)).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
for epoch in range(5):
    running_loss = 0.0
    for imgs, lbls in train_loader:
        imgs, lbls = imgs.to(device), lbls.to(device)
        optimizer.zero_grad()
        outputs = model(imgs)
        loss = criterion(outputs, lbls)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()
    print(f"Epoch {epoch+1}, Loss: {running_loss/len(train_loader):.4f}")
model.eval()
correct = total = 0
with torch.no_grad():
    for imgs, lbls in test_loader:
        imgs, lbls = imgs.to(device), lbls.to(device)
        _, pred = torch.max(model(imgs), 1)
```

```
        correct += (pred == lbls).sum().item()
        total += lbls.size(0)
print(f"Test Accuracy: {100*correct/total:.2f}%")
imgs, lbls = next(iter(test_loader))
imgs, lbls = imgs[:5].to(device), lbls[:5].to(device)
pred = model(imgs).argmax(1)
plt.figure(figsize=(10, 5))
for i in range(5):
    plt.subplot(1, 5, i+1)
    plt.imshow(imgs[i].cpu().squeeze(), cmap="gray")
    plt.title(f"P:{target_names[pred[i]]}\nT:{target_names[lbls[i]]}")
    plt.axis("off")
plt.show()
```

OUTPUT:

Epoch 1, Loss: 1.7634
Epoch 2, Loss: 1.7317
Epoch 3, Loss: 1.7319
Epoch 4, Loss: 1.7343
Epoch 5, Loss: 1.7347
Test Accuracy: 45.34%



RESULT:

EXP NO	LANGUAGE MODELING USING RNN	DATE
4		

AIM:

To write a python program for language modeling using RNN.

ALGORITHM:

Step 1: Start the program.
Step 2: Input the training text as a sequence of words.
Step 3: Build the vocabulary by extracting all unique words from the text and assigning an index to each word.
Step 4: Prepare input sequences by taking every group of three consecutive words as input and the following word as the target.
Step 5: Convert the input words and target words into numerical form using the word-to-index mapping.
Step 6: Convert the prepared data into tensors so that it can be used for training.
Step 7: Define the RNN model with an embedding layer to represent words, an RNN layer to learn word sequences, and a fully connected output layer with softmax activation for next-word prediction.
Step 8: Set up the training process by selecting cross-entropy as the loss function and Adam as the optimizer.
Step 9: Train the model for multiple epochs by feeding input sequences, calculating loss, and updating weights through backpropagation until the model learns the word relationships.
Step 10: For prediction, take a seed text such as “where are we”, convert it into indices, and pass it through the trained model to predict the most likely next word.
Step 11: Display the predicted word as output.
Step 12: End the program.

PROGRAM:

```
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.preprocessing import OneHotEncoder
text = "where are we going where are we meeting where are we now"
words = text.split()
vocab = sorted(set(words))
word2idx = {w: i for i, w in enumerate(vocab)}
idx2word = {i: w for w, i in word2idx.items()}
vocab_size = len(vocab)
sequences = []
for i in range(3, len(words)):
    seq = words[i-3:i+1]   # 3 input words + 1 target
    sequences.append(seq)
X, y = [], []
for seq in sequences:
    X.append([word2idx[w] for w in seq[:-1]])
    y.append(word2idx[seq[-1]])
X = np.array(X)  # shape (samples, 3)
y = np.array(y)  # shape (samples,)
X_tensor = torch.tensor(X, dtype=torch.long)
y_tensor = torch.tensor(y, dtype=torch.long)
class RNNModel(nn.Module):
    def __init__(self, vocab_size, embed_size, hidden_size):
        super(RNNModel, self).__init__()
        self.embed = nn.Embedding(vocab_size, embed_size)
        self.rnn = nn.RNN(embed_size, hidden_size, batch_first=True)
        self.fc = nn.Linear(hidden_size, vocab_size)
```

```
    def forward(self, x):
        x = self.embed(x)
        out, _ = self.rnn(x)
        out = self.fc(out[:, -1, :])  # use last time step
        return out
model = RNNModel(vocab_size, embed_size=10, hidden_size=64)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.01)
epochs = 500
for epoch in range(epochs):
    optimizer.zero_grad()
    outputs = model(X_tensor)
    loss = criterion(outputs, y_tensor)
    loss.backward()
    optimizer.step()
    if (epoch+1) % 100 == 0:
        print(f"Epoch {epoch+1}/{epochs}, Loss: {loss.item():.4f}")
def predict_next_word(seed_text):
    tokens = [word2idx[w] for w in seed_text.split()]
    tokens = torch.tensor(tokens, dtype=torch.long).unsqueeze(0)
    with torch.no_grad():
        output = model(tokens)
        predicted_index = torch.argmax(output, dim=1).item()
    return idx2word[predicted_index]
seed_text = "where are we"
predicted_word = predict_next_word(seed_text)
print(f"\nInput: '{seed_text}'")
print(f"Predicted next word: '{predicted_word}'")
```

OUTPUT:

Epoch 100/500, Loss: 0.3663
Epoch 200/500, Loss: 0.3662
Epoch 300/500, Loss: 0.3662
Epoch 400/500, Loss: 0.3662
Epoch 500/500, Loss: 0.3662

Input: 'where are we'
Predicted next word: 'going'

RESULT:

EXP NO	SENTIMENT ANALYSIS USING LSTM	DATE
5		

AIM:

To write a python program for sentiment analysis using LSTM.

ALGORITHM:

Step 1: Start the program.

Step 2: Define a small dataset of sentences with their sentiment labels (1 = Positive, 0 = Negative).

Step 3: Build a vocabulary by assigning an index to each unique word. Reserve <PAD> for padding and <UNK> for unknown words.

Step 4: Convert each sentence into a sequence of word indices, and pad them to a fixed length.

Step 5: Define the LSTM model with:

- Embedding layer (to convert words into vectors)
- LSTM layer (to capture word sequence patterns)
- Fully connected layer with Sigmoid activation (for binary output).

Step 6: Compile the model using Binary Cross-Entropy loss and Adam optimizer.

Step 7: Train the model for a fixed number of epochs, updating weights after each pass over the dataset.

Step 8: Test the model by giving a new review sentence, encode it, and predict whether the sentiment is Positive or Negative.

Step 9: Display the predicted sentiment and probability score.

Step 10: Stop the program.

PROGRAM:

```
import torch
import torch.nn as nn
import torch.optim as optim
sentences = ["i loved the movie", "this film was great", "fantastic story",
             "i hated the movie", "this film was boring", "terrible story"]
labels = [1, 1, 1, 0, 0, 0]  # 1=positive, 0=negative
word2idx = {"<PAD>": 0, "<UNK>": 1}
for s in sentences:
    for w in s.split():
        if w not in word2idx:
            word2idx[w] = len(word2idx)
def encode(s, max_len=5):
    ids = [word2idx.get(w, 1) for w in s.split()]
    return ids[:max_len] + [0]*(max_len-len(ids))
X = torch.tensor([encode(s) for s in sentences])
y = torch.tensor(labels, dtype=torch.float32)
class SentimentLSTM(nn.Module):
    def __init__(self, vocab, embed=8, hidden=16):
        super().__init__()
        self.embed = nn.Embedding(vocab, embed)
        self.lstm = nn.LSTM(embed, hidden, batch_first=True)
        self.fc = nn.Linear(hidden, 1)
    def forward(self, x):
        x = self.embed(x)
        _, (h, _) = self.lstm(x)
        return torch.sigmoid(self.fc(h[-1]))
model = SentimentLSTM(len(word2idx))
loss_fn = nn.BCELoss()
opt = optim.Adam(model.parameters(), lr=0.01)
```

```
for epoch in range(10):
    opt.zero_grad()
    out = model(X).squeeze()
    loss = loss_fn(out, y)
    loss.backward(); opt.step()
    print(f"Epoch {epoch+1}, Loss={loss:.4f}")
def predict(s):
    with torch.no_grad():
        out = model(torch.tensor([encode(s)]))
    return "Positive" if out.item() >= 0.5 else "Negative", out.item()
review = "i loved the film"
sentiment, score = predict(review)
print(f"\nReview: {review}\nSentiment: {sentiment}, Score={score:.4f}")
```

OUTPUT:

Epoch 1, Loss=0.6831
Epoch 2, Loss=0.6731
Epoch 3, Loss=0.6620
Epoch 4, Loss=0.6494
Epoch 5, Loss=0.6343
Epoch 6, Loss=0.6158
Epoch 7, Loss=0.5929
Epoch 8, Loss=0.5647
Epoch 9, Loss=0.5306
Epoch 10, Loss=0.4906

Review: i loved the film
Sentiment: Positive, Score=0.6390

RESULT:

EXP NO	PARTS OF SPEECH TAGGING USING SEQUENCE TO SEQUENCE ARCHITECTURE	DATE
6		

AIM:

To write a python program for speech tagging using sequence to sequence architecture.

ALGORITHM:

Step 1: Start the program.
Step 2: Import necessary libraries (nltk, collections, random).
Step 3: Download NLTK datasets treebank, punkt, and punkt_tab.
Step 4: Load the POS-tagged sentences from the NLTK Treebank corpus.
Step 5: Split data into a training set to learn word–tag patterns and a test set to evaluate accuracy.
Step 6: Build word–tag frequency counts from the training set. For each word, count how many times each POS tag occurs. For each tag, count how often it follows another tag (tag bigram counts).
Step 7: Define the encoder function which converts a sentence into lowercase words (tokens).
Step 8: Define the decoder function. For each word, choose the most likely POS tag based on word-to-tag frequency and previous tag (tag bigram) probability. Continue until all words in the sentence are tagged.
Step 9: Encode a new sentence using the encoder.
Step 10: Decode the sentence using the decoder to get predicted POS tags.
Step 11: Evaluate on the test set by comparing predicted tags with true tags and computing accuracy as (correct tags / total tags) × 100%.
Step 12: Display the sentence, words, predicted POS tags, and accuracy.
Step 13: Stop the program.

PROGRAM:

```
import nltk
nltk.download("punkt")
nltk.download("punkt_tab")
nltk.download("treebank")
from nltk.corpus import treebank
from collections import defaultdict
import random
nltk.download("treebank")
nltk.download("punkt")
tagged_sents = treebank.tagged_sents()[:2000]  # use first 2000 for training
test_sents = treebank.tagged_sents()[2000:2050]  # use 50 for testing
word_tag_freq = defaultdict(lambda: defaultdict(int))
tag_bigram_freq = defaultdict(lambda: defaultdict(int))
for sent in tagged_sents:
    prev_tag = "<SOS>"
    for word, tag in sent:
        word = word.lower()
        word_tag_freq[word][tag] += 1
        tag_bigram_freq[prev_tag][tag] += 1
        prev_tag = tag
    tag_bigram_freq[prev_tag]["<EOS>"] += 1
def encoder(sentence):
    """Encodes words into lowercase tokens."""
    return [w.lower() for w in nltk.word_tokenize(sentence)]
def decoder(words):
    """Decodes words into POS tags using most likely sequence (greedy)."""
    tags = []
    prev_tag = "<SOS>"
    for w in words:
```

```
        if w in word_tag_freq:
            # choose tag with max likelihood (word+context)
            tag_probs = {t: word_tag_freq[w][t] * (tag_bigram_freq[prev_tag][t] + 1)
                         for t in word_tag_freq[w]}
            best_tag = max(tag_probs, key=tag_probs.get)
        else:
            # unseen word → choose most frequent tag after prev_tag
            best_tag = max(tag_bigram_freq[prev_tag], key=tag_bigram_freq[prev_tag].get)
        tags.append(best_tag)
        prev_tag = best_tag
    return tags
sentence = "The quick brown fox jumps over the lazy dog ."
encoded = encoder(sentence)
decoded_tags = decoder(encoded)
print("Sentence:", sentence)
print("Words:", encoded)
print("Predicted POS Tags:", decoded_tags)
correct, total = 0, 0
for sent in test_sents:
    words = [w.lower() for w, t in sent]
    true_tags = [t for w, t in sent]
    pred_tags = decoder(words)
    for t, p in zip(true_tags, pred_tags):
        total += 1
        if t == p:
            correct += 1
print(f"\nTest Accuracy: {correct/total:.2%}")
```

INSTALLATION:

```
pip install torch torchvision torchaudio
pip install nltk
```

OUTPUT

```
[nltk_data] Downloading package punkt to C:\Users\win
[nltk_data]     10\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to C:\Users\win
[nltk_data]     10\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping tokenizers\punkt_tab.zip.
[nltk_data] Downloading package treebank to C:\Users\win
[nltk_data]     10\AppData\Roaming\nltk_data...
[nltk_data]   Package treebank is already up-to-date!
[nltk_data] Downloading package treebank to C:\Users\win
[nltk_data]     10\AppData\Roaming\nltk_data...
[nltk_data]   Package treebank is already up-to-date!
[nltk_data] Downloading package punkt to C:\Users\win
[nltk_data]     10\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
Sentence: The quick brown fox jumps over the lazy dog .
Words: ['the', 'quick', 'brown', 'fox', 'jumps', 'over', 'the', 'lazy', 'dog', '.']
Predicted POS Tags: ['DT', 'JJ', 'NNP', 'NNP', 'NNP', 'IN', 'DT', 'NN', 'IN', '.']

Test Accuracy: 82.60%
```

RESULT:

EXP NO	MACHINE TRANSLATION USING ENCODER-DECODER MODEL	DATE
7		

AIM:

To write a python program for machine translation using Encoder-Decoder Model.

ALGORITHM:

Step 1: Start the program.
Step 2: Import necessary libraries (random).
Step 3: Define a small dataset of English–French sentence pairs.
Step 4: Build an English-to-French dictionary from the dataset: for each English word, store its corresponding French word(s).
Step 5: Define the encoder function that converts an input English sentence into a list of lowercase words (tokens).
Step 6: Define the decoder function that maps each English token to its French translation using the dictionary, choosing randomly if multiple translations exist, and using <UNK> for unknown words.
Step 7: Define the translate function that passes the English sentence through the encoder, then through the decoder, and joins the decoded tokens to form the French sentence.
Step 8: Test the program with example English sentences by encoding, decoding, and printing the French translation.
Step 9: Stop the program.

PROGRAM:

```
import random
pairs = [
    ('hello', 'bonjour'),
```

```
    ('how are you', 'comment allez vous'),
    ('i am fine', 'je vais bien'),
    ('thank you', 'merci'),
    ('good morning', 'bonjour'),
    ('good night', 'bonne nuit'),
    ('see you', 'à bientôt'),
    ('yes', 'oui'),
    ('no', 'non'),
    ('please', 's’il vous plaît')
]
eng2fra = {}
for eng, fra in pairs:
    eng_tokens = eng.lower().split()
    fra_tokens = fra.lower().split()
    for e_word, f_word in zip(eng_tokens, fra_tokens):
        if e_word not in eng2fra:
            eng2fra[e_word] = [f_word]
        elif f_word not in eng2fra[e_word]:
            eng2fra[e_word].append(f_word)

def encode(sentence):
    """Converts sentence to list of lowercase words (tokens)."""
    return sentence.lower().split()
def decode(tokens):
    """Maps English tokens to French tokens using dictionary."""
    translated = []
    for t in tokens:
        if t in eng2fra:
            # Choose randomly if multiple translations exist
            translated.append(random.choice(eng2fra[t]))
        else:
```

```
            translated.append("<UNK>")  # unknown word
    return translated
def translate(sentence):
    tokens = encode(sentence)
    fra_tokens = decode(tokens)
    return ' '.join(fra_tokens)
test_sentences = ["hello", "how are you", "good night", "please", "see you"]
for s in test_sentences:
    print(f"{s} -> {translate(s)}")
```

OUTPUT:

hello -> bonjour
how are you -> comment allez vous
good night -> bonne nuit
please -> s’il
see you -> à vous

RESULT:

EXP NO	IMAGE AUGMENTATION USING GANS	DATE
8		

AIM:

To write a python program for Image Augmentation using GANS.

ALGORITHM:

Step 1: Start the program.
Step 2: Import the required libraries such as torch, torchvision, and matplotlib.
Step 3: Load the MNIST dataset and apply normalization.
Step 4: Define the Generator network that takes random noise and produces an image.
Step 5: Define the Discriminator network that classifies an image as real or fake.
Step 6: Initialize both networks, define the loss function, and select optimizers.
Step 7: For each epoch, train the Discriminator with real and fake images.
Step 8: Train the Generator so that it produces images that can fool the Discriminator.
Step 9: After fixed intervals, generate augmented images from the Generator and display them.
Step 10: Repeat the process for the required number of epochs.
Step 11: Stop training and display the message that augmentation is complete.

PROGRAM:

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
```

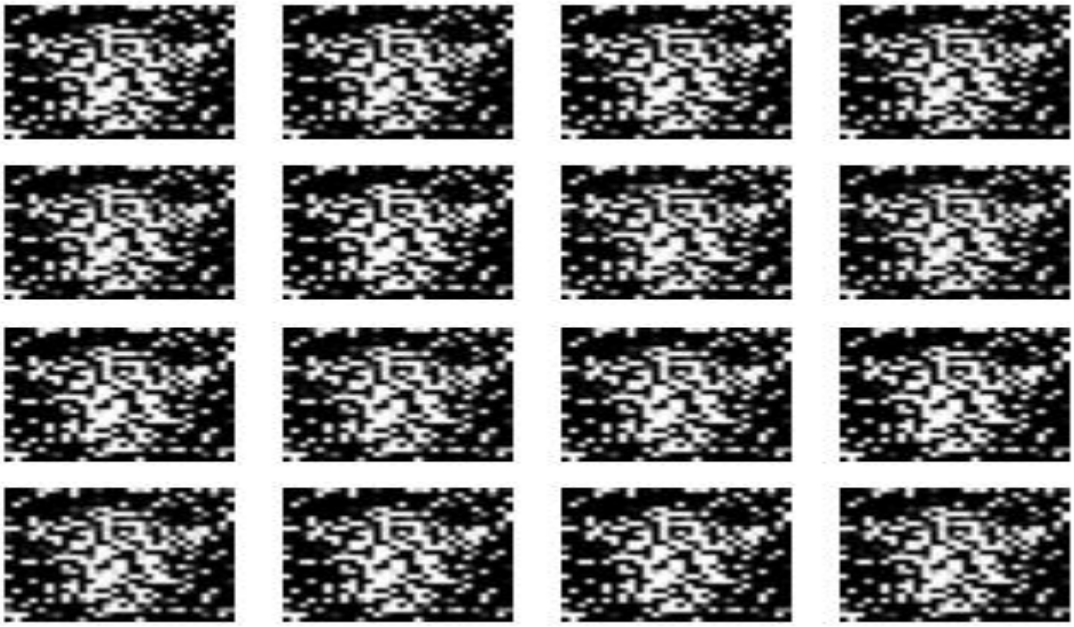
```
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])
trainset = torchvision.datasets.MNIST(
    root='./data', train=True, download=True, transform=transform
)
subset, _ = torch.utils.data.random_split(trainset, [5000, len(trainset)-5000])
trainloader = torch.utils.data.DataLoader(subset, batch_size=64, shuffle=True)
class Generator(nn.Module):
    def __init__(self, latent_dim=64):
        super().__init__()
        self.model = nn.Sequential(
            nn.Linear(latent_dim, 128),
            nn.ReLU(),
            nn.Linear(128, 256),
            nn.ReLU(),
            nn.Linear(256, 28*28),
            nn.Tanh()
        )
    def forward(self, z):
        img = self.model(z)
        return img.view(z.size(0), 1, 28, 28)
class Discriminator(nn.Module):
    def __init__(self):
        super().__init__()
        self.model = nn.Sequential(
            nn.Flatten(),
            nn.Linear(28*28, 256),
            nn.LeakyReLU(0.2),
            nn.Linear(256, 1),
```

```
            nn.Sigmoid()
        )
    def forward(self, img):
        return self.model(img)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
latent_dim = 64
generator = Generator(latent_dim).to(device)
discriminator = Discriminator().to(device)
criterion = nn.BCELoss()
optimizer_G = optim.Adam(generator.parameters(), lr=0.001)
optimizer_D = optim.Adam(discriminator.parameters(), lr=0.001)
epochs = 50   # fast training
for epoch in range(1, epochs+1):
    for imgs, _ in trainloader:
        imgs = imgs.to(device)
        batch_size = imgs.size(0)
        real_labels = torch.ones(batch_size, 1).to(device)
        fake_labels = torch.zeros(batch_size, 1).to(device)
        optimizer_D.zero_grad()
        loss_real = criterion(discriminator(imgs), real_labels)
        z = torch.randn(batch_size, latent_dim).to(device)
        fake_imgs = generator(z)
        loss_fake = criterion(discriminator(fake_imgs.detach()), fake_labels)
        loss_D = (loss_real + loss_fake) / 2
        loss_D.backward()
        optimizer_D.step()
        optimizer_G.zero_grad()
        loss_G = criterion(discriminator(fake_imgs), real_labels)
        loss_G.backward()
        optimizer_G.step()
```

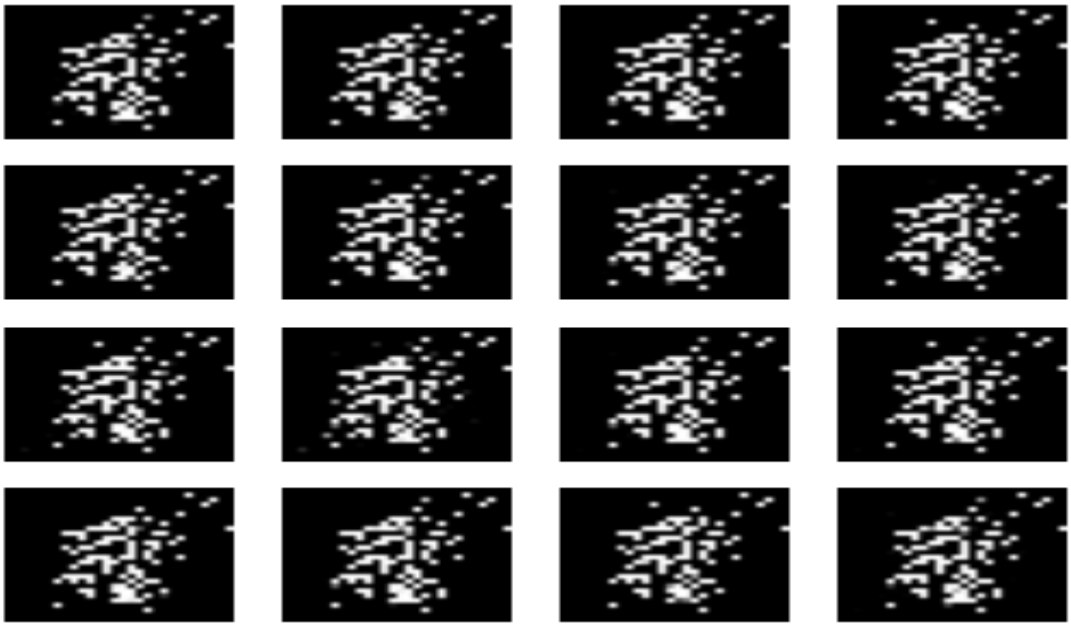
```
    if epoch % 10 == 0:
        print(f"Epoch {epoch} | D loss: {loss_D.item():.4f} | G loss: {loss_G.item():.4f}")
        with torch.no_grad():
            sample_z = torch.randn(16, latent_dim).to(device)
            gen_imgs = generator(sample_z).cpu()
            gen_imgs = 0.5 * gen_imgs + 0.5  # Scale to [0,1]
            fig, axs = plt.subplots(4, 4, figsize=(4,4))
            cnt = 0
            for i in range(4):
                for j in range(4):
                    axs[i, j].imshow(gen_imgs[cnt, 0, :, :], cmap="gray")
                    axs[i, j].axis("off")
                    cnt += 1
            plt.show()
print("Training complete! Displayed augmented images during training.")
```

OUTPUT:

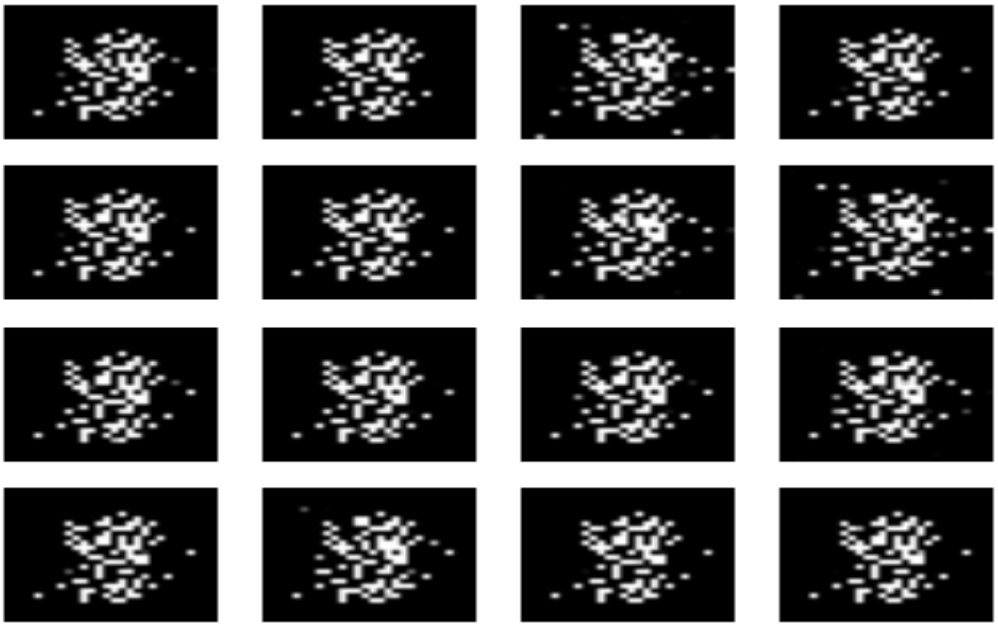
Epoch 10 | D loss: 0.0200 | G loss: 7.7106



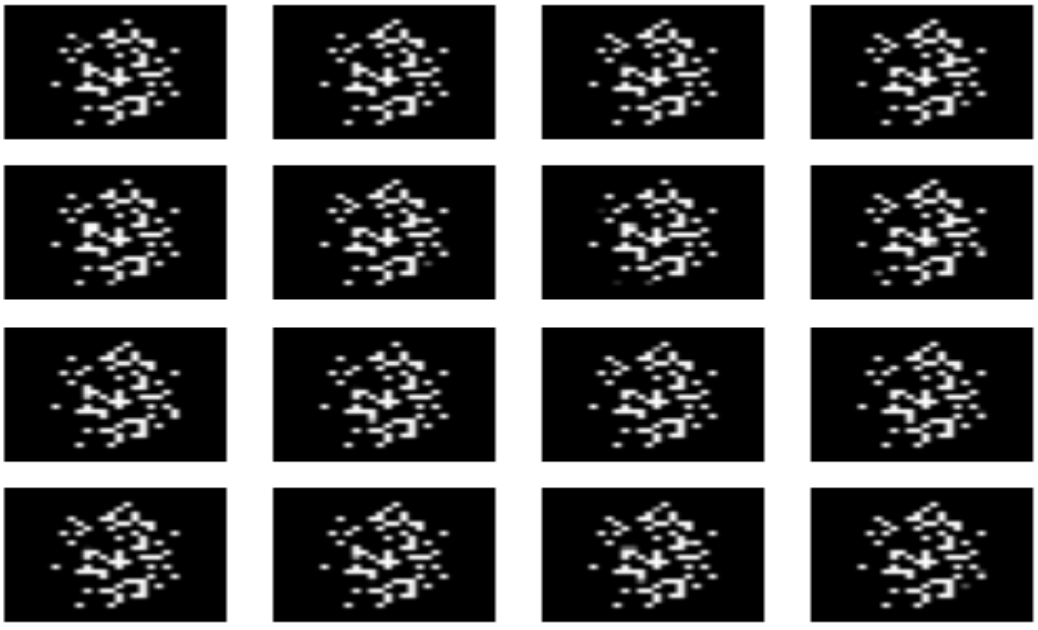
Epoch 20 | D loss: 0.0095 | G loss: 5.9044



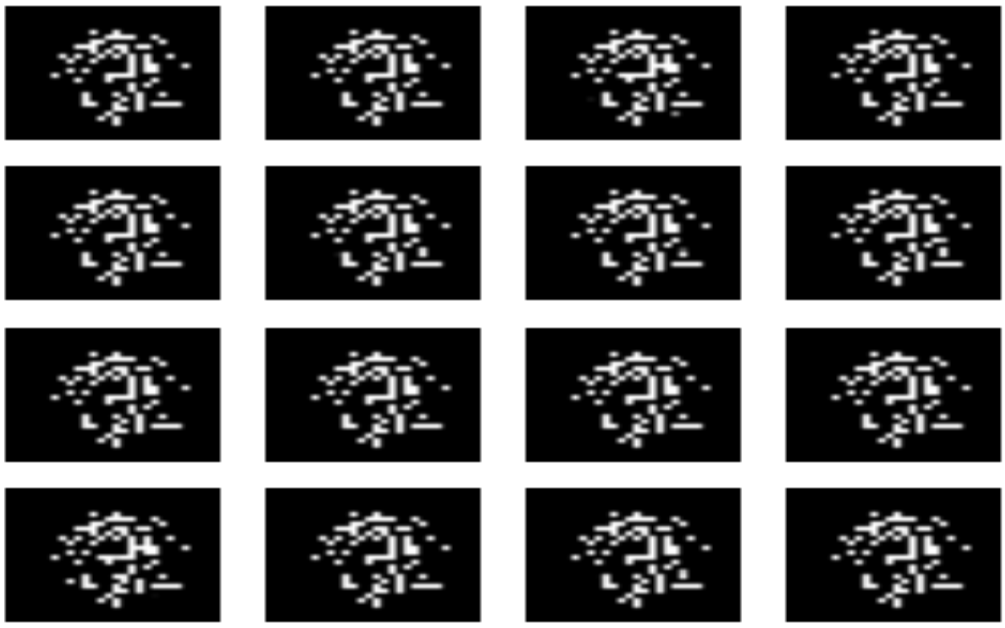
Epoch 30 | D loss: 0.0011 | G loss: 8.1043



Epoch 40 | D loss: 0.0006 | G loss: 7.5293



Epoch 50 | D loss: 0.0001 | G loss: 8.6463



RESULT: